SYN Flood Attack Investigation Using tshark

OJE DOMINION MAYOWA

FWSD2511507

Basic Networking Skills for Digital Forensics

MR. Aminu Idris

11 September 2026

LAB OVERVIEW

This lab compared a normal HTTP handshake against a bounded SYN probe simulation to identify indicators of incomplete or suspicious TCP activity. I captured a legitimate curl-based session as a baseline, then used a Scapy script to send four raw SYN packets in a controlled loopback environment, and analyzed both captures using tshark filters, counts, and expert diagnostics.

OBJECTIVES

By the end of this exercise, the learner will be able to:

- Differentiate a complete TCP three-way handshake from an incomplete or half-open connection attempt.
- Apply tshark display filters and field extraction to identify SYN, SYN-ACK, ACK, and RST packets within a capture.
- Calculate key indicators of anomalous connection behavior, including SYN counts, unique source ports, response ratios, and incomplete-handshake occurrences.
- Construct a controlled, four-packet loopback-only traffic simulation using Scapy, strictly within an authorized lab environment.
- Develop a defensible, evidence-based timeline and incident finding derived from PCAP analysis.
- Recommend appropriate detection and mitigation controls for SYN flood activity based on observed traffic patterns.

TOOLS

- Authorized training Evidence
- Kali Linux
- Microsoft Word
- Github, Git
- Md5sum and sha256sum
- Apache2
- Wireshark, tshark, Web browser, curl
- python3-scapy

METHODOLOGY

Lab Folder Structure and Evidence Preparation

The first step is to create a working environment, I created the folder structure before downloading or generating evidence of any form.

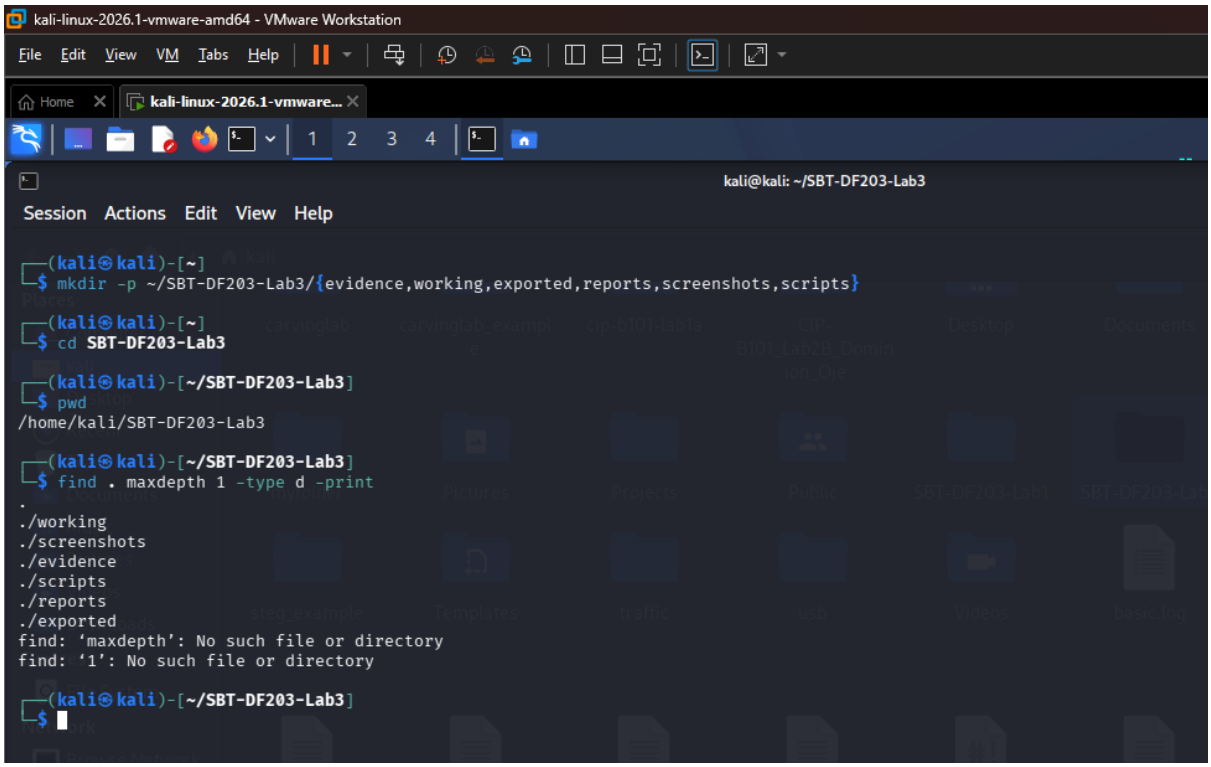

Fig 1: Creating the working environment

Now we then generate evidence and download the necessary tools needed to run the lab and also getting the packets that would be used for the SYN-Flood attack from the link and using the command

wget -O evidence/mySYNFloodCapture.pcap 'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap'

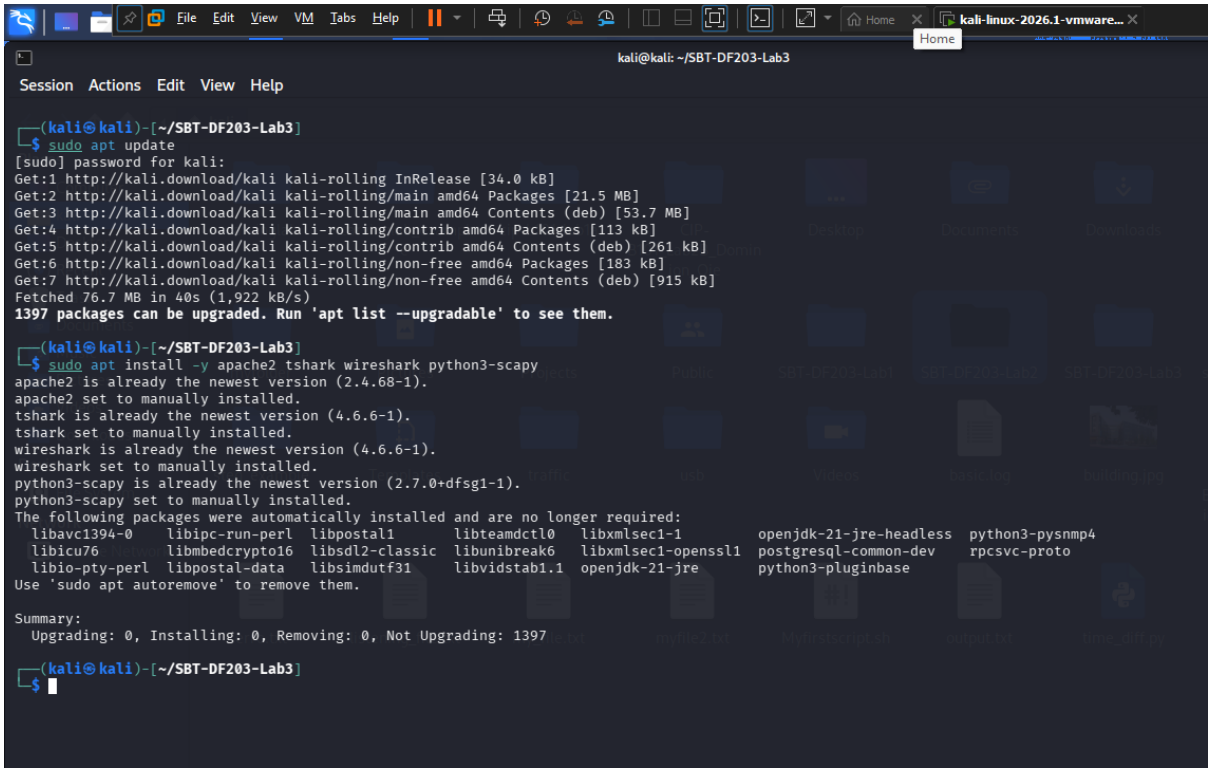

Fig 2: Updating the terminal and installing necessary tools

Fig 3: Apache starts in the background, and the packets are downloaded

Then we check the Integrity by hashing the value of both the original evidence and the copy and from the image we see that the hash value is same for both the original copy and the working copy. The original copy was kept in the evidence folder and the copy for analysis was kept in the working folder.

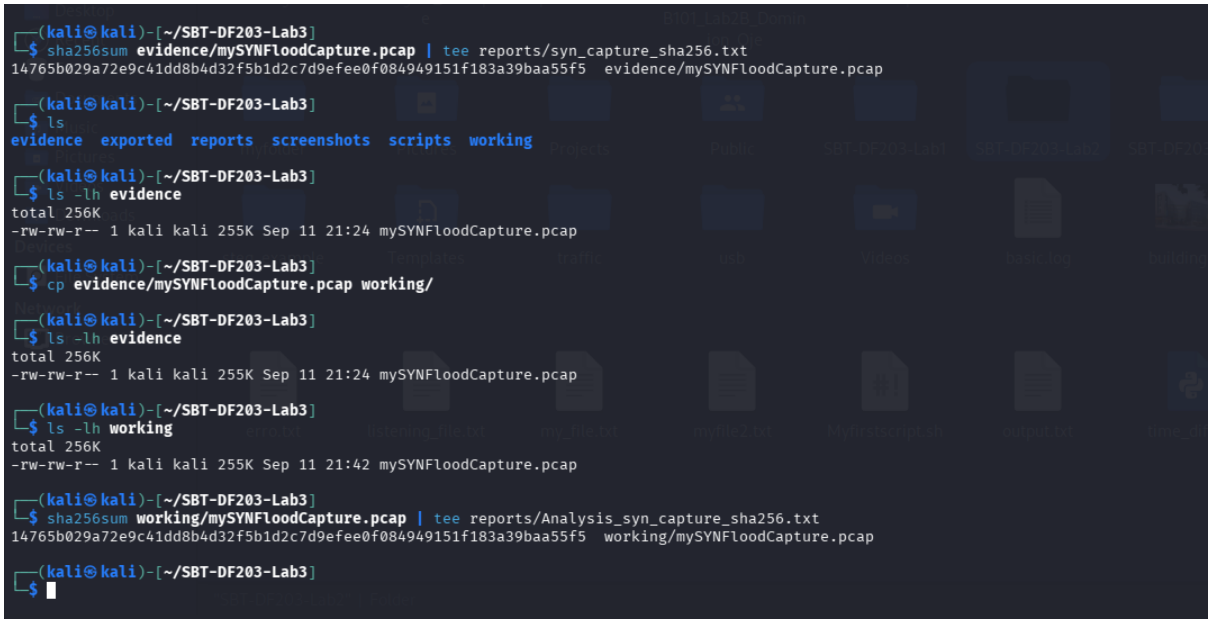

Fig 4: Hashing the original and copy and checking their differences

Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab3-Oje-Dominion-Mayowa
Trainee name	OJE DOMINION MAYOWA
Date and time started	11/09/2026, 8:00PM
Evidence file name(s)	mySYNFloodCapture.pcap,
Source or generation method	Github link,
Original SHA-256	**14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5**
Working-copy SHA-256	**14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5**
Analysis workstation/VM	Kali Linux
Notes on any changes	

Part A - Establish a Normal Handshake Baseline

The first step was to start the capture on a separate terminal (terminal 1)

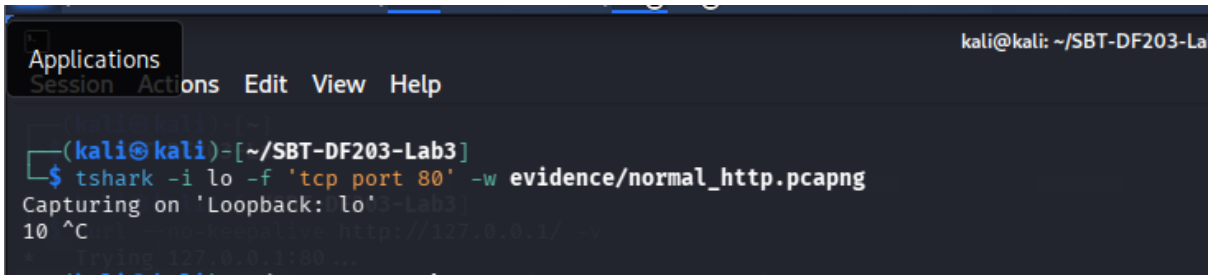

Fig 5: Start capturing or listening with tshark

Then we run the curl to run a normal handshake on another terminal (terminal 2)

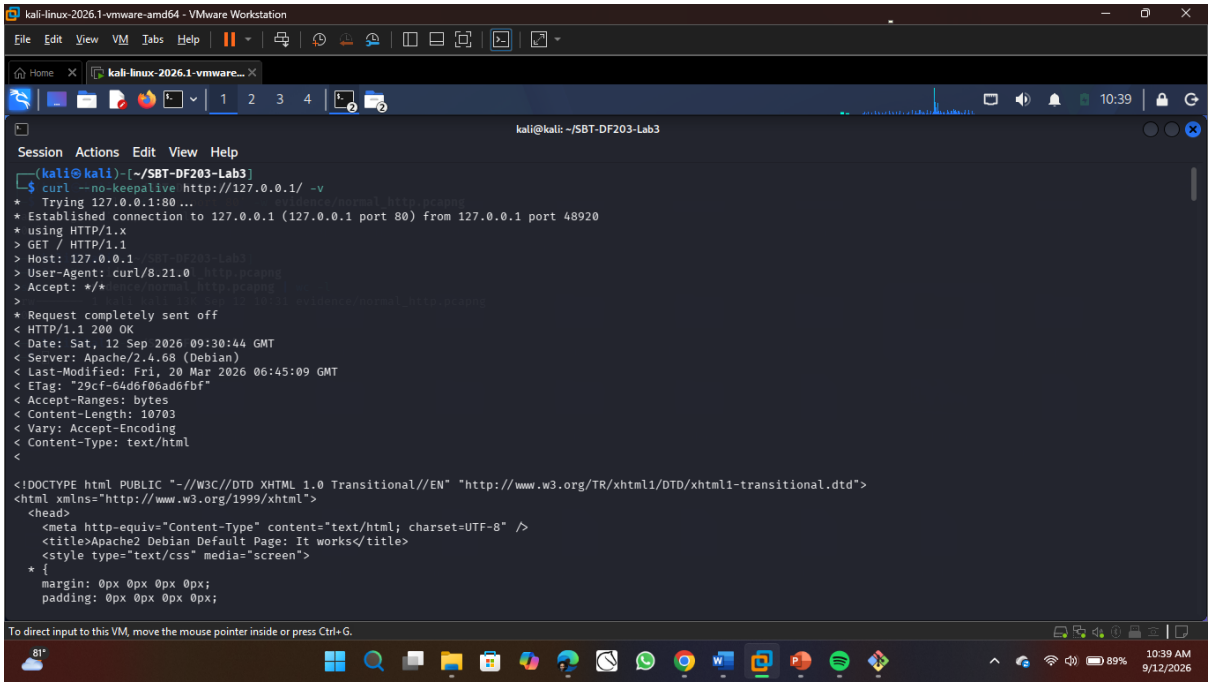

Fig 6: Sending an HTTP requests using curl

Fig 7: The handshake response as seen

A complete, healthy TCP connection follows this pattern: SYN → SYN-ACK → ACK to open the connection, followed later by FIN-ACK → FIN-ACK → ACK to close it. In this baseline capture, the handshake opened correctly (packets 1–2, with the plain ACK as packet

3 not shown here since it lacks SYN/FIN flags), and the connection was closed cleanly on both sides (packets 8–9), confirming normal, expected TCP behavior with no dropped, retransmitted, or half-open connections.

Part B - Conduct the Bounded Loopback Simulation

The first step was to create the scapy script to be able to fire 4 fake TCP connection-opening packets (SYN packets) to my local Apache server, and then stops. It doesn't wait for replies, neither does it complete any handshakes, and it doesn't keep running.

Fig 8: Scapy Script

Then using tshark we created a listener to capture anything going on in port 80

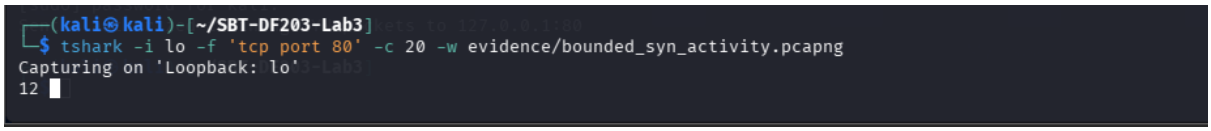

Fig 9: Started capture in terminal 1

Then the script was run and it was successful as it only sent four packets to the IP on the port

Fig 10: Successfully ran the Scapy script

Part C - Identify SYN Indicators with tshark

We then indentify SYN indicators with tshark and then we check the hashes of both the original variable in the evidence folder and the one in the working folder. From the image below we see that they are exactly the same.

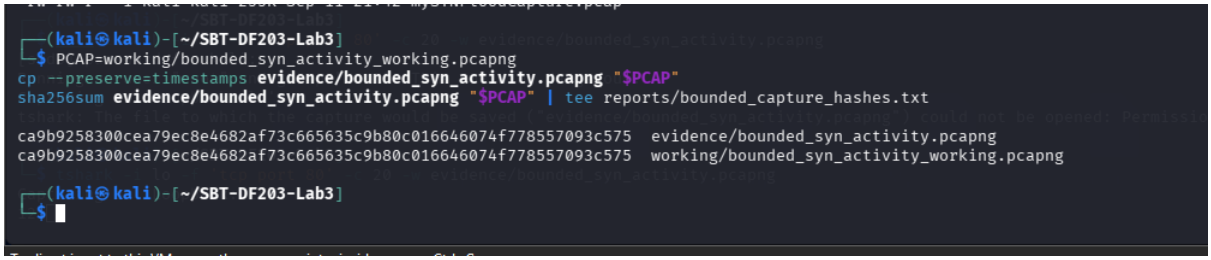

Fig 11: Making a copy of the evidence of the SYN FLOOD

Starting with the Initial Sequences (SYN's) only with the frame number (1,4,7,10)

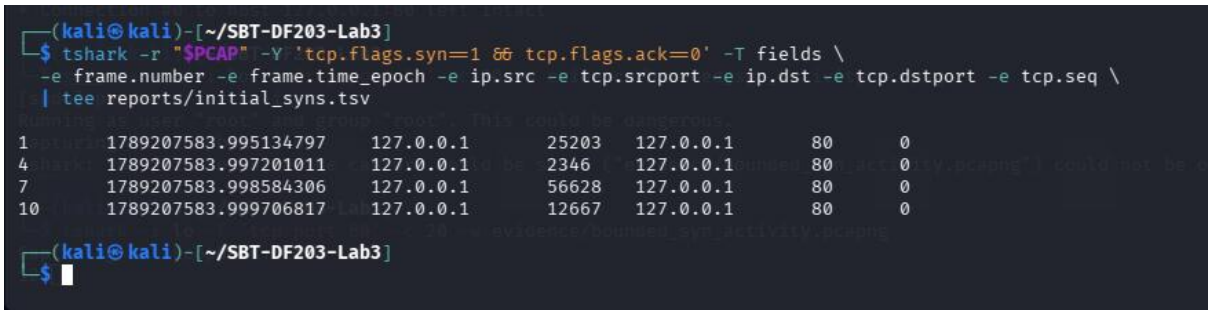

Fig 12: Starting with the Initial SYNs being listed

Now we get the SYN-ACK responses, that is for every SYN request there is a SYN-ACK response which have the frame number (2,5,8,11)

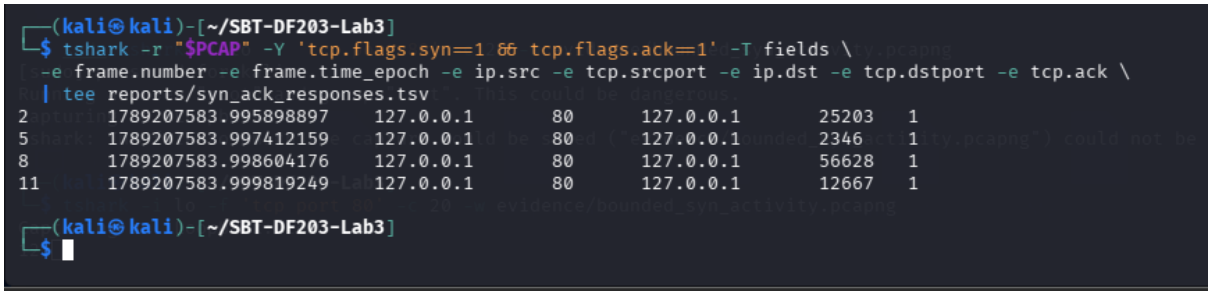

Fig 13: SYN-ACK responses

Then we get a final ACK response which search through the capture file for packets that look like responses to the SYN probes,

Fig 14: ACK Responses with a Reset Flag

Part D - Quantify and Correlate the Activity

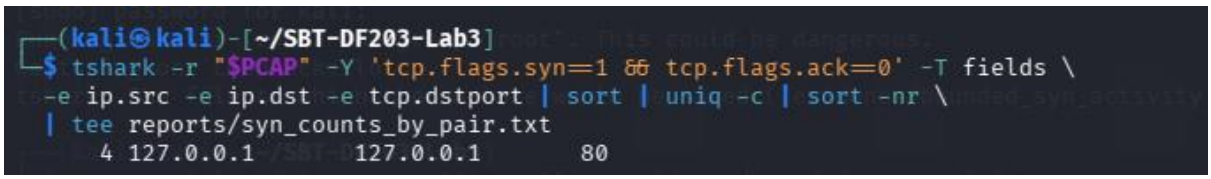

Fig 15: Number of Bare SYNs

This counts how many bare SYNs (no ACK) came from each source to each destination/port.

Fig 16: Displays Unique Client source port

The image above lists each distinct source port used which shows 4 different random ports, one per SYN packet (since the script used RandShort()).

Then we save the reports and this pulls Wireshark's built-in "expert analysis" flags anomalies like retransmissions, malformed packets, or unusual behavior automatically.

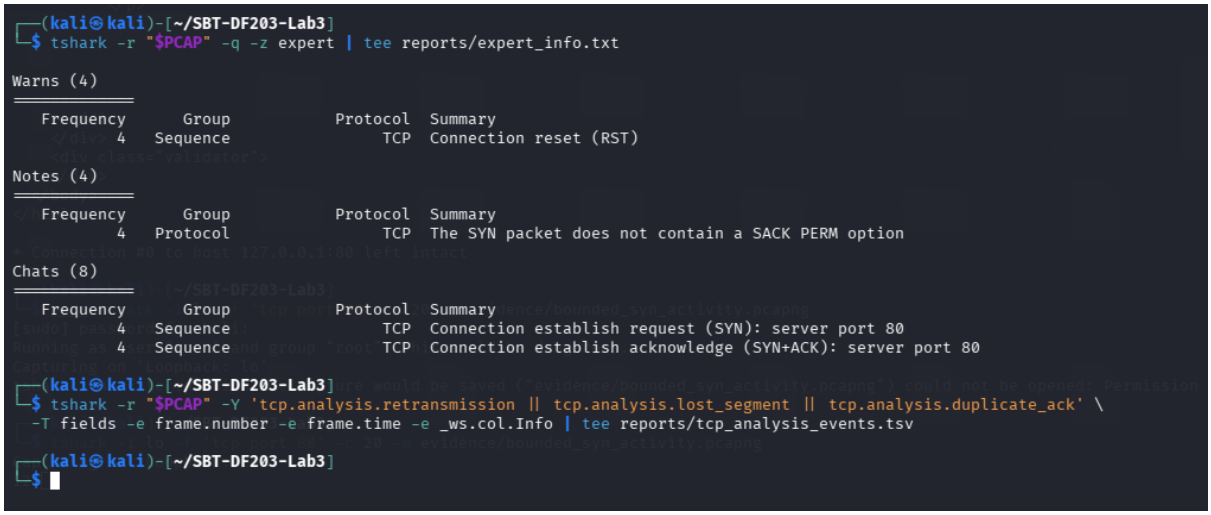

Fig 17: Gets a Built-in report from wireshark

Part E - Compare Normal and Suspicious Sessions

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
Initial SYN count	1	4	Multiple SYNs from the same source to the same destination in a tight time window (all within ~4 milliseconds) is a strong indicator of scripted/automated activity rather than normal user behavior
SYN-ACK count	1	4	The server responded correctly to every SYN, confirming Apache itself was healthy and behaving normally; the anomaly originates from the client side, not the server
Completed handshakes	1 (full 3-way handshake)	0	None of the 4 connections were completed with a final ACK; every one

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
			was aborted with a RST instead, indicating half-open, abandoned connection attempts rather than legitimate sessions
Unique client source ports	1 (48920)	4 (25203, 2346, 56628, 12667)	Rapidly cycling through many distinct source ports toward the same destination:port is a classic signature of scripted connection attempts, such as scanning or flood-style activity
HTTP request present?	Yes (GET / returned full page)	No	No application-layer data was ever exchanged; the activity never advanced beyond the TCP handshake stage, confirming these were not genuine client requests
RST packets	0	4 (frames 3, 6, 9, 12)	Every connection attempt was actively reset immediately after the server's SYN-ACK; this is consistent with the source machine never intending to complete a real connection, a key fingerprint of the tooling used (Scapy raw packets bypassing the kernel's TCP stack)
Observed duration	~12ms across handshake+close	~4ms across all 4 SYN/SYN-ACK/RST cycles (timestamps 995 to 999 in the same second)	Extremely tight timing between 4 separate "connection attempts" from different ports is inconsistent with real user behavior and consistent with automated packet generation

From what I observed, the normal session went through a full handshake and an actual HTTP exchange, while the SYN probe activity showed four handshakes that never completed, each from a different source port, and each one ending in a RST packet, all happening within milliseconds of each other. This made it easy to see the clear difference between real client traffic and scripted, aborted connection attempts.